Detailed Line-by-Line Explanation of Parallel BFS and DFS Code

This document explains each line of the uploaded C++ program that implements Parallel Breadth First Search (BFS) and Parallel Depth First Search (DFS) using OpenMP.

Line	Code	Explanation
1	#include <iostream>	Includes the iostream library for input and output operations.

Line	Code	Explanation
2	#include <vector>	Includes the vector library to use dynamic arrays.

Line	Code	Explanation
3	#include <queue>	Includes the queue library to implement BFS traversal.

Line	Code	Explanation
4	#include <omp.h>	Includes the OpenMP library for parallel programming.

Line	Code	Explanation
5	using namespace std;	Allows use of standard library names without std:: prefix.

Line	Code	Explanation
6		Blank line for readability.

Line	Code	Explanation
7	void parallelBFS (int start, vector <vector <int>> &adj, int n)	Defines the parallel BFS function.

Line	Code	Explanation
8	{	Beginning of BFS function body.

Line	Code	Explanation
9	vector<bool> visited(n,false);	Creates a visited array initialized to false.

Line	Code	Explanation
10	queue<int> q;	Creates a queue to store nodes during BFS.

Line	Code	Explanation
11		Blank line.

Line	Code	Explanation
12	visited[start] = true;	Marks the start node as visited.

Line	Code	Explanation
13	q.push(start);	Pushes the start node into the queue.

Line	Code	Explanation
14		Blank line.

Line	Code	Explanation
15	cout<<"Parallel BFS Traversal :";	Prints BFS traversal heading.

Line	Code	Explanation
16		Blank line.

Line	Code	Explanation
17	while (!q.empty())	Loop continues until the queue becomes empty.

Line	Code	Explanation
18	{	Start of while-loop block.

Line	Code	Explanation
19	int node = q.front();	Gets the front node from the queue.

Line	Code	Explanation
20	q.pop();	Removes the front node from the queue.

Line	Code	Explanation
21	cout<<node<<" ";	Prints the current node.

Line	Code	Explanation
22		Blank line.

Line	Code	Explanation
23	#pragma omp parallel for	Parallelizes the upcoming for-loop using OpenMP.

Line	Code	Explanation
24	for (int i=0;i<adj[node].size();i++)	Loops through all neighbors of the current node.

Line	Code	Explanation
25	{	Start of for-loop block.

Line	Code	Explanation
26	int neighbor = adj[node][i];	Stores the current neighbor node.

Line	Code	Explanation
27		Blank line.

Line	Code	Explanation
28	if (!visited[neighbor])	Checks whether the neighbor is unvisited.

Line	Code	Explanation
29	{	Start of if block.

Line	Code	Explanation
30	#pragma omp critical	Critical section ensures only one thread modifies shared data at a

Line	Code	Explanation
31	if (!visited[neighbor])	Double-checks if neighbor is still unvisited.

Line	Code	Explanation
32	{	Start of critical block.

Line	Code	Explanation
33	visited[neighbor] = true;	Marks neighbor as visited.

Line	Code	Explanation
34	q.push(neighbor);	Adds neighbor to the BFS queue.

Line	Code	Explanation
35	}	End of critical block.

Line	Code	Explanation
36	}	End of if block.

Line	Code	Explanation
37	}	End of for-loop block.

Line	Code	Explanation
38	}	End of while-loop block.

Line	Code	Explanation
39	cout<<endl;	Prints a newline after BFS traversal.

Line	Code	Explanation
40	}	End of BFS function.

Line	Code	Explanation
41		Blank line.

Line	Code	Explanation
42	void parallelDFS (int node, vector <vector <int>> &adj, vector <int> &visited)	Defines the parallel DFS function.

Line	Code	Explanation
43	{	Beginning of DFS function body.

Line	Code	Explanation
44	visited[node] = true;	Marks current node as visited.

Line	Code	Explanation
45	cout<<node<<" ";	Prints current node.

Line	Code	Explanation
46		Blank line.

Line	Code	Explanation
47	#pragma omp parallel for	Parallelizes neighbor traversal.

Line	Code	Explanation
48	for (int i=0; i<adj[node].size(); i++)	Loops through all neighbors of current node.

Line	Code	Explanation
49	{	Start of loop block.

Line	Code	Explanation
50	int neighbor = adj[node][i];	Stores neighbor node.

Line	Code	Explanation
51	if (!visited[neighbor])	Checks if neighbor is unvisited.

Line	Code	Explanation
52	{	Start of if block.

Line	Code	Explanation
53	#pragma omp task	Creates a parallel task for recursive DFS call.

Line	Code	Explanation
54	if (!visited[neighbor])	Double-checks if neighbor remains unvisited.

Line	Code	Explanation
55	{	Start of task block.

Line	Code	Explanation
56	parallelDFS(neighbor, adj, visited);	Recursively performs DFS on neighbor.

Line	Code	Explanation
57	}	End of task block.

Line	Code	Explanation
58	}	End of if block.

Line	Code	Explanation
59	}	End of loop block.

Line	Code	Explanation
60	}	End of DFS function.

Line	Code	Explanation
61		Blank line.

Line	Code	Explanation
62	int main()	Main function starts execution of program.

Line	Code	Explanation
63	{	Beginning of main function body.

Line	Code	Explanation
64	int n = 6;	Creates a graph with 6 nodes.

Line	Code	Explanation
65	vector <vector <int>> adj(n);	Creates adjacency list representation of graph.

Line	Code	Explanation
66	vector <int> visited (n,0);	Creates visited array for DFS traversal.

Line	Code	Explanation
67	adj[0] = {1,2};	Adds edges for node 0.

Line	Code	Explanation
68	adj[1] = {0,3,4};	Adds edges for node 1.

Line	Code	Explanation
69	adj[2] = {0,5};	Adds edges for node 2.

Line	Code	Explanation
70	adj[3] = {1};	Adds edge for node 3.

Line	Code	Explanation
71	adj[4] = {1};	Adds edge for node 4.

Line	Code	Explanation
72	adj[5] = {2};	Adds edge for node 5.

Line	Code	Explanation
73		Blank line.

Line	Code	Explanation
74	parallelBFS (0, adj, n);	Calls parallel BFS starting from node 0.

Line	Code	Explanation
75		Blank line.

Line	Code	Explanation
76	cout<<"Parallel DFS Traversal : ";	Prints DFS traversal heading.

Line	Code	Explanation
77	#pragma omp parallel	Starts parallel OpenMP region.

Line	Code	Explanation
78	{	Beginning of parallel block.

Line	Code	Explanation
79	#pragma omp single	Ensures only one thread initiates DFS.

Line	Code	Explanation
80	{	Beginning of single block.

Line	Code	Explanation
81	parallelDFS (0, adj, visited);	Calls DFS starting from node 0.

Line	Code	Explanation
82	}	End of single block.

Line	Code	Explanation
83	}	End of parallel block.

Line	Code	Explanation
84		Blank line.

Line	Code	Explanation
85	cout<<endl;	Prints newline after DFS traversal.

Line	Code	Explanation
86	return 0;	Returns 0 indicating successful execution.

Line	Code	Explanation
87	}	End of main function.

Summary: The program demonstrates parallel graph traversal using OpenMP. BFS uses a queue and parallel loop processing, while DFS uses recursive tasks for parallel execution.